Operating Systems & Security Lab: The Quantum Vault Deadlock

Duration: 3 Hours Topic: File System Mounting, Process Synchronization, and Deadlocks

Key Learning Objectives

By the end of this lab, students will be able to:

1. **Provision Virtual Storage**: Use standard Linux utilities (`dd` , `mkfs`) to create and format virtual disk image files.
2. **Manage File Systems**: Understand and execute loopback device mounting and safe unmounting processes without root privileges.
3. **Analyze Deadlocks**: Intentionally trigger a classic OS Deadlock (Circular Wait) using concurrent background processes.
4. **Implement Synchronization**: Use Resource Ordering strategies to resolve deadlocks and ensure safe concurrent execution.
5. **Cross-User Collaboration**: Work with a partner to simulate distributed system deadlocks across different user environments on the same server.
6. **Implement Deadlock Recovery**: Utilize timeouts and preemption logic (`flock -w`) to gracefully abort locked processes.
7. **Trace Processes**: Use system monitoring tools to identify hung processes and forcefully terminate them.

Mission Briefing (Disaster Recovery)

Welcome back to QuantumTech. After securing the widget purchasing system, you have been reassigned to the Disaster Recovery (DR) team.

QuantumTech stores its most critical intellectual property across two offline, encrypted virtual drives: Vault Alpha and Vault Beta. To ensure high availability, the DR system runs automated bash scripts to bi-directionally sync data between the two vaults.

However, there is a massive flaw in the current prototype. When the sync scripts run concurrently, the entire DR server freezes. Your mission is to provision the virtual drives, reproduce the server freeze locally, scale the attack to a distributed cross-user deadlock, and finally rewrite the scripts using safe synchronization principles.

IMPORTANT SYSTEM NOTES:

- Environment: Log in to the provided shared Linux server using your student account.

- Privileges & Mounting: Standard `mount` commands require root (`sudo`). Because your student account operates with standard privileges, you will use `udisksctl` (a user-space disk management tool) to mount the drives safely.
- Workspace Organization (Dual Folders): 1. Scripts: You must save all your bash scripts strictly inside your `~/bin` directory. 2. Data & Mounts: You must create a new directory in your home folder called `~/os-lab-deadlock` . All virtual drives, mount points, lock files, and your `README.md` report must be kept inside this folder.
- Execution: Because your `~/bin` folder is already included in your system's `PATH` , you can execute your scripts directly by name from anywhere without the `./` prefix.
- File Naming Convention: All scripts must strictly follow standard Linux naming conventions: lowercase letters, words separated by underscores (`snake_case`), and no file extensions.
- Git Naming Convention: You will eventually turn your `~/os-lab-deadlock` folder into a Git repository and push it. Rename it to `os-lab-deadlock-<student_id>` when it is time to submit.

Task Overview

Here is a quick summary of the 7 levels you will complete during this lab:

- Level 1: Virtual Vault Provisioning - Create, format, and mount two virtual disk images.
- Level 2: The Naive Sync - Write two concurrent scripts that attempt to lock and access both vaults.
- Level 3: The Local Deadlock - Execute the scripts simultaneously and document the server freeze.
- [15-MINUTE BREAK] - Recommended break point before multiplayer collaboration.
- Level 4: Site-to-Site Sync (Multiplayer Deadlock) - Pair up with a classmate and deadlock each other across the server.
- Level 5: Global Resource Ordering (The Patch) - Rewrite the scripts to request resources in a strict global order.
- Level 6: Deadlock Recovery (The Timeout Patch) - Write a script that gracefully aborts if a lock takes too long to acquire.
- Level 7: Safe Ejection - Write a `teardown` script to safely unmount the file systems and release the loopback devices.

Target File Structure

By the end of this lab, your home directory should be organized exactly like this structure (depending on if you are Player A or B in Level 4):

```
~
├── bin/                        # ALL Executable Scripts go here
│   ├── cross_sync_alpha        # (Or cross_sync_beta) Script from Level 4
│   ├── sync_down               # Script from Level 2 & 5
│   ├── sync_timeout            # Script from Level 6
│   ├── sync_up                 # Script from Level 2 & 5
│   └── teardown                # Script from Level 7
│
└── os-lab-deadlock/            # ALL Data and Mounts go here
    ├── mount_alpha             # Symlink to your actual Vault Alpha mount
    ├── mount_beta              # Symlink to your actual Vault Beta mount
    ├── public_dr_alpha/        # (Or public_dr_beta) DMZ dir from Level 4
    │   └── vault.lock          # Cross-user lock file
    ├── README.md               # Final Markdown deliverable with screenshots &
    ├── vault_alpha.img         # Virtual drive image from Level 1
    └── vault_beta.img          # Virtual drive image from Level 1
```

Core Concepts & Warm-Up

Before diving into the Disaster Recovery server implementation, let's review the fundamental operating system concepts you will be exploiting and fixing.

1. File System Mounting, Virtual Drives, & The `sudo` Constraint

Concept: In Linux, storage devices (like USBs or hard drives) are not automatically assigned letters (like `C:\` or `D:\`). Instead, they must be "mounted" or attached to an existing empty directory in the file system tree.

A Virtual Drive (Loopback Device) allows us to treat a regular file (like a blank `.img` file full of zeroes) as if it were a physical block device (hard drive). This is perfect for shared servers where you can't plug in a physical USB without conflicting with other students.

The `sudo` Constraint: Traditionally, the `mount` command modifies the kernel's global Virtual File System (VFS) tree. Because a malicious mount can compromise the entire server and affect all users, `mount` strictly requires root (`sudo`) privileges. Since you are operating as a standard user without `sudo` , you cannot use the traditional `mount` command. Instead, you will use User-Space Mounting tools like `udisksctl` . This tool talks to a background service (`udisks2`) that is specifically authorized by the OS to safely set up loop devices and mount them for non-root users without compromising the entire server's security.

Example A: Traditional Physical Mount (Requires `sudo`)

```
# Used by sysadmins for physical USBs or hard
drives mkdir /mnt/external_us
sudo mount /dev/sdb1 /mnt/external_usb
sudo umount /mnt/external_usb
```

Example B: User-Space Virtual Mount (No `sudo` Required - Used in this Lab)

```
# Used by standard users to mount virtual image
files # 1. Attach the file to a virtual loop device
udisksctl loop-setup -f my_virtual_drive.img

# 2. Tell the OS to mount it (The OS will assign it a path like
/run/media/user/. udisksctl mount -b /dev/loop0

# 3. Safely unmount and detach the loop device when
finished

udisksctl unmount -b /dev/loop0
udisksctl loop-delete -b /dev/loop0
```

2. Process Synchronization & Mutex

Concept: When multiple processes run concurrently, they might try to read and write to the same file at the exact same time, causing data corruption (a Race Condition). A Mutex (Mutual Exclusion) uses file locks to force processes to wait in line.

Mutex Example:

```
# flock ensures only one process can enter the critical section at a
time (
flock -x 200
echo "I have exclusive access to the
file!" sleep 2
) 200> shared_data.lock
```

3. Deadlock & Circular Wait

Concept: A deadlock is a system state where two or more processes are frozen indefinitely because they are each waiting for a resource held by the other.

Analogy: Process A holds the "Pen" and needs the "Paper" to continue. Process B holds the "Paper" and needs the "Pen" to continue. Neither process will let go of what they have until they get what they need. Both are stuck forever.

Deadlock Logic Example:

```
# Process 1                      # Process 2

flock -x 200 (Locks Pen)         flock -x 201 (Locks
Paper) sleep 1                   sleep 1
flock -x 201 (Waits for Paper)   flock -x 200 (Waits for

Pen) # RESULT: Both processes freeze forever.
```

The Fix (Prevention via Resource Ordering): If we enforce a global rule that says *every process must grab the Pen before they are allowed to grab the Paper*, the deadlock is mathematically impossible. Process B will be forced to wait for the Pen before it ever picks up the Paper, allowing Process A to finish its work and release both.

4. Deadlock Recovery (Timeouts & Preemption)

Concept: Sometimes you cannot perfectly order all resources in a complex distributed system. Instead of *preventing* deadlocks entirely, we implement Deadlock Recovery. If a process suspects it is stuck in a deadlock (usually by waiting too long), it aborts, releases its current locks (preemption), and frees the system so other processes can continue.

Analogy: If Process B waits for the "Paper" for more than 5 minutes, it assumes a deadlock has occurred. It gives up, drops the "Pen" on the desk so someone else can use it, and tries the whole task again later.

Recovery Logic Example:

```
# flock -w tells the command to Wait a maximum number of seconds, then fail grace (
echo "Attempting to acquire lock..."
if flock -w 5 200; then
    echo "Success! Processing data..."
    sleep 2
else
    echo "[ERROR] Timeout reached. Aborting to recover from potential deadlock."
fi
) 200> shared_data.lock
```

5. Command Arsenal (Know Your Tools)

You will use several powerful Linux system utilities in this lab. Do not just copy and paste; understand what you are telling the kernel to do:

- `dd` (Data Duplicator): Used to copy and convert raw data. We use it to create empty image files.
  - `if=/dev/zero` : Input File. Tells the tool to read an infinite stream of raw, empty zeroes.
  - `of=<file>` : Output File. Where to write the zeroes (your virtual drive image).
  - `bs=1M` : Block Size. Writes data in chunks of 1 Megabyte.
  - `count=10` : How many blocks to write (e.g., 10 blocks of 1M = 10MB total size).
- `mkfs.ext4` : Make File System. Formats a raw device or image file into the `ext4` Linux file system so it can understand directories and files.
  - `-F` : Force. Bypasses the warning prompt telling you it isn't a physical hard drive.
- `udisksctl` : User-space disk management tool. Safely performs disk operations for standard users.
  - `loop-setup -f <file>` : Attaches an image file to a virtual `/dev/loop` device.
  - `mount -b <device>` : Tells the OS to mount the virtual block device.
  - `unmount -b <device>` : Unmounts the block device from the file system.
  - `loop-delete -b <device>` : Detaches the image file from the loop device, freeing the kernel resource.
- `ln -s` : Creates a symbolic link (a shortcut) from one file path to another.
- `flock` (File Lock): Manages file locks from shell scripts.
  - `-x` : Acquires an exclusive lock (only one process can hold it at a time).

- `-w <seconds>` : Wait/Timeout. Aborts the attempt if the lock cannot be acquired within the specified number of seconds.

Level 1: Virtual Vault Provisioning (Formatting & Mounting)

Before we can sync data, we need virtual drives. We will create these data files in the new `os-lab-deadlock` directory.

Execute & Verify Tasks:

1. Workspace Setup: Create your script and data directories.
   ```
   mkdir -p ~/bin
   mkdir -p ~/os-lab-deadlock
   cd ~/os-lab-deadlock
   ```

2. Allocate Space: Use the `dd` command to create two blank 10MB image files.

   ```
   dd if=/dev/zero of=vault_alpha.img bs=1M count=10
   ```

   ```
   dd if=/dev/zero of=vault_beta.img bs=1M count=10
   ```

3. Format the Drives: Build an `ext4` file system on them (using `-F` to force execution on a file). `mkfs.ext4 -F vault_alpha.img mkfs.ext4 -F vault_beta.img`
4. Attach the Virtual Drives: Use the user-space disk tool to create loop devices. `udisksctl loop-setup -f vault_alpha.img udisksctl loop-setup -f vault_beta.img` *(Observe the output. It will say something like "Mapped file... to /dev/loop0". Note which loop number belongs to Alpha and Beta).*
5. Mount the Drives: Now, mount those specific loop devices. `udisksctl mount -b /dev/loop0` *(Replace 0 with your Alpha loop number)* `udisksctl mount -b /dev/loop1` *(Replace 1 with your Beta loop number)*
6. Create the Symlinks: As a standard user, `udisksctl` mounted your drives to a complex path like `/media/username/uuid-string` . To make our scripts clean, we will create symlinks (shortcuts) to those mounts right inside our lab folder.
   - Look at the output from step 5. It will say "Mounted /dev/loopX at `/run/media/...` "
   - Create the links: `ln -s /run/media/.../alpha-uuid-here mount_alpha ln -s /run/media/.../beta-uuid-here mount_beta`

Observation Checkpoint 1: Run `df -h | grep loop` . You should see your loopback devices mounted. Take a screenshot of the output, embed it in your `README.md` file, and write a 1-2 sentence explanation of what the output proves regarding the loop devices.

Level 2: The Naive Sync (Introducing the Flaw)

We have two synchronization scripts. `sync_up` pulls data from Alpha and writes to Beta.

`sync_down` pulls from Beta and writes to Alpha.

Create the script named `sync_up` inside your `~/bin` folder (`chmod +x ~/bin/sync_up`):

```
(
  echo "[Sync_UP] Locking Vault
  Alpha..."
  flock -x 200
  echo "[Sync_UP] Vault Alpha locked. Computing data delta..."
  sleep 2
  echo "[Sync_UP] Locking Vault Beta to write..."
  flock -x 201
  echo "[Sync_UP] Vault Beta locked.
  Syncing data..."
  sleep 1
  echo "[Sync_UP] Sync complete. Releasing locks."
) 200>$HOME/os-lab-deadlock/mount_alpha/lockfile
201>$HOME/os-lab-deadlock/mount_beta/lockfile
```

Create the script named `sync_down` inside your `~/bin` folder (`chmod +x ~/bin/sync_down`):

```
(
  echo "[Sync_DOWN] Locking Vault Beta..."
  flock -x 201
  echo "[Sync_DOWN] Vault Beta locked. Computing data
  delta..." sleep 2

  echo "[Sync_DOWN] Locking Vault Alpha to
  write..." flock -x 200
  echo "[Sync_DOWN] Vault Alpha locked. Syncing data..."
  sleep 1
  echo "[Sync_DOWN] Sync complete. Releasing locks."
)
200>$HOME/os-lab-deadlock/mount_alpha/lockfile
201>$HOME/os-lab-deadlock/mount_beta/lockfile
```

Level 3: The Local Circular Wait (Triggering Deadlock)

In Operating Systems, a Deadlock occurs when four conditions are met: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. Let's trigger a Circular Wait locally first.

Execute & Verify Tasks:

1. Open two separate SSH terminal windows connected to the server.
2. In Terminal 1, run `sync_up` .
3. Immediately switch to Terminal 2 and run `sync_down` (you must do this within 2 seconds).

Observation Checkpoint 2:

1. Observe the output in both terminals. The scripts will hang indefinitely. Neither will ever reach "Sync complete."
2. Open a third terminal. Run `ps aux | grep sync_` to see the stuck processes.
3. Terminate the hung processes by pressing `Ctrl+C` in Terminal 1 and Terminal 2.
4. Analysis: Embed a screenshot of the frozen terminals in your `README.md` . Below the image, explain exactly why the scripts froze. Reference which script held which lock, and

which lock they were waiting for.

BREAK POINT (15 Minutes) You have reached the halfway mark of the 3-hour lab. **This is a great time to stretch, grab a drink, and reset before moving into the multiplayer collaboration and patching phases!**

Level 4: Site-to-Site Sync (Multiplayer Deadlock)

A local deadlock is bad, but a distributed deadlock across different users is a disaster. You must now configure Linux permissions to expose your lockfiles to a classmate and simulate a cross-server synchronization failure.

Execute & Verify Tasks (Team Up!):

1. Find a Partner: Pair up with a classmate who is also logged into the shared server. Decide who will be Site Alpha (Player A) and who will be Site Beta (Player B).
2. The Public DMZ: Both players need to create a public synchronization directory *inside* their `~/os-lab-deadlock` folder and grant permissions so the partner can read and lock files inside it.
   - Player A (Alpha):

     ```
     mkdir ~/os-lab-deadlock/public_dr_alpha
     touch ~/os-lab-deadlock/public_dr_alpha/vault.lock
     chmod o+x ~                        # Allows traversing through your home
     chmod  o+rx  ~/os-lab-deadlock     # Allows executing/reading your data
     chmod o+rx ~/os-lab-deadlock/public_dr_alpha
     chmod o+rw ~/os-lab-deadlock/public_dr_alpha/vault.lock
     ```

   - Player B (Beta):

     ```
     mkdir ~/os-lab-deadlock/public_dr_beta
     touch ~/os-lab-deadlock/public_dr_beta/vault.lock
     chmod o+x ~
     chmod o+rx ~/os-lab-deadlock
     chmod o+rx ~/os-lab-deadlock/public_dr_beta
     chmod o+rw ~/os-lab-deadlock/public_dr_beta/vault.lock
     ```

3. The Multiplayer Scripts: You will write a script in `~/bin` that locks your *own* vault locally,

and then reaches across the server to lock your *partner's* vault using their absolute path.

- Player A writes `cross_sync_alpha` (in `~/bin`):
  - Locks `$HOME/os-lab-deadlock/public_dr_alpha/vault.lock` (Local)
  - Sleeps for 2 seconds
  - Locks `/home/<PLAYER_B_USERNAME>/os-lab-deadlock/public_dr_beta/vault.lock`
- Player B writes `cross_sync_beta` (in `~/bin`):
  - Locks `$HOME/os-lab-deadlock/public_dr_beta/vault.lock` (Local)
  - Sleeps for 2 seconds
  - Locks `/home/<PLAYER_A_USERNAME>/os-lab-deadlock/public_dr_alpha/vault.lock`

4. Fire: Both of you run your scripts at the *exact same time*.

Observation Checkpoint 3: Look at your terminal. You should both be completely frozen. You have successfully deadlocked two separate user accounts on the server! Press `Ctrl+C` to break the lock. Take a screenshot of your frozen terminal script output, embed it in your `README.md`, and briefly explain how this simulates a distributed denial of service.

Level 5: Global Resource Ordering (The Patch)

To prevent Deadlocks in distributed systems, one of the four necessary conditions must be broken. The most reliable method is breaking the Circular Wait.

We do this using Resource Ordering: We mandate that all processes across the *entire system/network* must acquire resources in the exact same strict, global order, regardless of what they intend to do with them.

Execute & Verify Tasks:

1. Agree on a Global Order: You and your partner must establish a rule. For example: "Alpha's lock must ALWAYS be acquired before Beta's lock."
2. Patch the Scripts: * Player A's script is already safe (it locks Alpha, then Beta).
   - Player B must rewrite `cross_sync_beta`. Even though Player B is pushing data *from* Beta *to* Alpha, they must acquire the lock for Alpha first, and Beta second.
3. Retest: Run the simultaneous execution test from Level 4 again.

Observation Checkpoint 4: Observe the terminals. Player A will acquire Alpha. Player B will attempt to acquire Alpha and safely pause (wait in queue). Player A will lock Beta, finish their sync, and release the locks. Player B will then automatically acquire both locks and proceed

safely.

Verify: Take a screenshot of the successful terminal outputs completing sequentially. Add this screenshot to your `README.md` file, and explain exactly how modifying the order of locks broke the Circular Wait condition.

Level 6: Deadlock Recovery (The Timeout Patch)

While Resource Ordering is clean, another valid OS technique is Deadlock Recovery via Timeouts. This breaks the "Infinite Wait" (No Preemption) condition. Instead of freezing forever, a script will wait a few seconds and gracefully abort if the resource is unavailable, freeing up system memory.

Execute & Verify Tasks:

1. Create a new script named `sync_timeout` inside your `~/bin` folder (`chmod +x ~/bin/sync_timeout`).
2. We will use `flock -w 5` , which tells the system to wait a maximum of 5 seconds for the lock.
3. Write the following logic into `sync_timeout` :

```
(
  echo "[Sync_Timeout] Attempting to lock Vault Alpha (Timeout:
  5s)..." if flock -w 5 200; then
      echo "[Sync_Timeout] Lock acquired safely! Syncing data..."
      sleep 2
      echo "[Sync_Timeout] Success. Releasing
  lock." else
      echo "[ERROR] Lock timeout! Aborting sync to prevent system deadlock."
  fi
) 200>$HOME/os-lab-deadlock/mount_alpha/lockfile
```

4. The Test: In Terminal 1, run your old `sync_up` script (which holds the Alpha lock for 3+ seconds). In Terminal 2, immediately run `sync_timeout` .

Observation Checkpoint 5: Observe Terminal 2. It will pause for 5 seconds, realize it cannot get the lock safely, and cleanly abort without freezing your terminal. Take a screenshot of the Timeout Error message, embed it in your `README.md` , and explain why this timeout strategy is useful for server health.

Level 7: Safe Ejection (Teardown)

Just like yanking a physical USB drive out of a computer can corrupt data, destroying the virtual images while they are mounted will corrupt the file system and leave orphaned loopback devices in the kernel.

Create a script named `teardown` in your `~/bin` folder (`chmod +x ~/bin/teardown`).

Execute & Verify Tasks:

Write a script that executes the following operations safely:

1. Print a message: "Unmounting virtual vaults..."
2. Unmount both loopback devices using the user-space utility. First, find which `/dev/loopX` your image is on by checking `losetup -a` . `udisksctl unmount -b /dev/loopX` *(Replace X with your actual loop numbers)*
3. Detach the loop devices from the image files to free up the kernel resources.

   ```
   udisksctl loop-delete -b /dev/loopX
   ```

4. Delete your shortcut symlinks from Level 1, and optionally clean up your `.img` files. `rm`

   ```
   ~/os-lab-deadlock/mount_alpha ~/os-lab-deadlock/mount_beta
   ```

Observation Checkpoint 6: Run `teardown` . Then run `df -h | grep loop` to prove the mounts are successfully removed. Take a screenshot of the clean `df -h` output, add it to your `README.md` , and explain why proper teardown is critical for system stability.

Final Deliverables

Note: You do not need to submit your actual script files. Your instructor will review and grade your scripts (`sync_up` , `sync_down` , `cross_sync_alpha` / `cross_sync_beta` , `sync_timeout` , `teardown`) directly within your `~/bin` folder on the shared Linux server. Please ensure they are saved with the correct names and are fully executable.

Push the contents of your `~/os-lab-deadlock` directory to your assigned Git repository. Your repository must be named according to the standard convention: `os-lab-deadlock-`